Weekly Project Progress-2

Group Work

Oussema Kirmani – 5153785

Gurpreet Singh Galsian– 5154655

Dhruv Dewan – 5150667

Data Analysis with Python

Algoma University

COSC-5806

Zamilur Rahman

March 15, 2026

Exploratory Data Analysis

Libraries

In this part of the project, we used the following Python libraries to examine the data and create graphs:

1. NumPy (Numerical Python)

NumPy is a Python library that allows you to perform math and work with arrays. It can manage large multidimensional arrays and mathematical functions.

2. Pandas

Pandas is a powerful Python library that enables you to manipulate and analyze data. It provides tools for cleaning, exploring, and modifying datasets using structures like DataFrames.

3. Scikit-learn (Sklearn)

Scikit-learn is an open-source machine learning library for Python. It offers effective tools for preprocessing, classification, regression, and dimensionality reduction in predictive data analysis.

4. Matplotlib

Matplotlib is a popular Python library for creating plots, graphs, and charts.

5. Seaborn

Seaborn is a Python data visualization library built on top of Matplotlib. It provides a sophisticated interface for creating appealing and informative statistical graphics.

Dataset Understanding

After loading the dataset, we used several commands to examine and understand its structure.

The following commands were used:

df.columns

df.info()

These commands helped identify the dataset's structure. There are 14 columns in the dataset, which include both numerical and categorical attributes. Heart Disease is the target variable, indicating whether a patient has heart disease.

Missing Value Analysis

We checked for missing values in the dataset using the following command:

df.isna().sum()

This command returns the number of missing values in each column.

Observation

The dataset does not have any missing values, indicating good data quality. Since there are no missing values, we did not need to perform any data cleaning.

Duplicate Record Analysis

To ensure data quality, we also checked for duplicate rows in the dataset using the following command:

df.duplicated().sum()

Observation

The dataset does not contain duplicate records. Each row represents a unique patient record, which is beneficial for accurate analysis and model training.

Class Distribution Analysis

We analyzed the distribution of the target variable using the following command:

df["Heart Disease"].value_counts()

Observation

The dataset appears to be slightly imbalanced, meaning the number of patients with heart disease differs slightly from those without it. Class imbalance can

impact machine learning models, as they may become biased toward the majority class. We will address this issue before training the machine learning models.

```
…  Heart Disease
   Absence     347546
   Presence    282454
   Name: count, dtype: int64
```

Visualization Settings

Before generating visualizations, we set the plotting style using the following commands:

sns.set_style("whitegrid")

plt.rcParams["figure.figsize"] = (10,6)

Explanation

sns.set_style("whitegrid")

This command sets the plot style to a white background with light grid lines, making the charts easier to read.

plt.rcParams["figure.figsize"] = (10,6)

This command sets the chart size to a width of 10 and height of 6, ensuring that visualizations are clear.

Target Variable Visualization

To visualize the distribution of heart disease cases, we used the following code:

sns.countplot(x="Heart Disease", data=df, palette="Set2")

plt.title("Heart Disease Class Distribution")

plt.show()

Explanation

countplot() counts how many times each category appears. It creates a bar chart showing the number of patients with and without heart disease. The palette="Set2" parameter selects a soft color scheme for the bars.

Observation

The visualization confirms that the dataset has a slight class imbalance between patients with heart disease and those without.

Numerical Feature Distribution Analysis

To understand how numerical health features vary between patients with and without heart disease, we created histograms for the following attributes:

- Age

- Cholesterol

- Max HR

- Number of vessels fluro

code:

sns.histplot(df, x=feature, hue="Heart Disease", kde=True)

Explanation

histplot() creates a histogram showing the distribution of data values. hue="Heart Disease" divides the data into two groups:

- Patients with heart disease

- Patients without heart disease

kde=True adds a smooth curve called Kernel Density Estimate, which indicates the general distribution pattern. alpha controls transparency to make both groups visible at the same time. bins controls the number of bars used in the histogram.

Observation

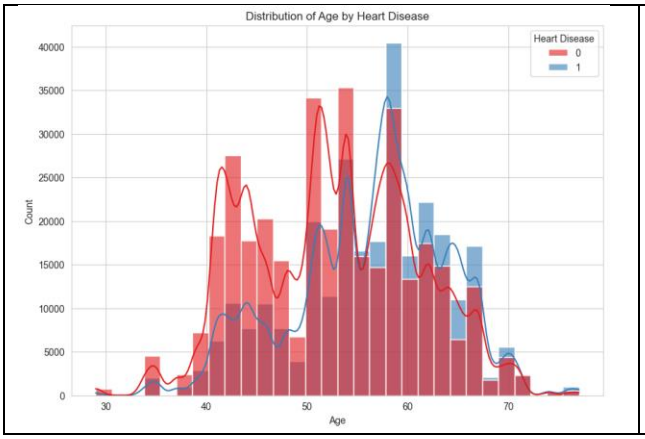
The distribution plots show that certain numerical features differ between patients with and without heart disease. In particular:

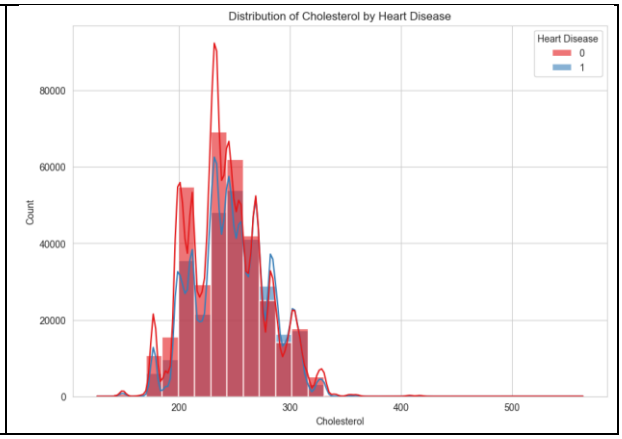
- Age distribution indicates that older individuals are more likely to have heart disease.

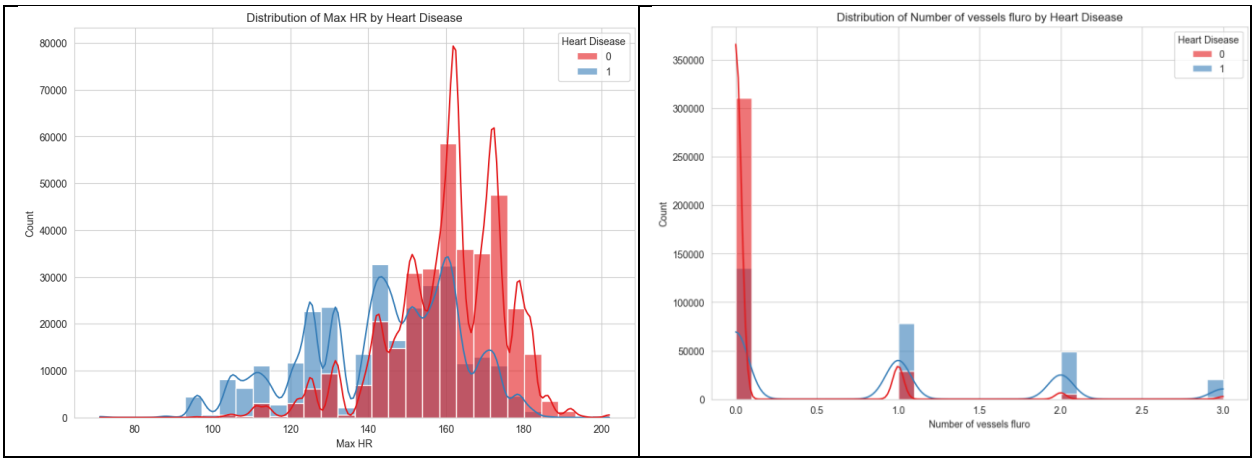
- Cholesterol levels seem higher in many heart disease cases.

- Maximum heart rate varies between the two groups.

- Patients with more blocked vessels tend to have a higher prevalence of heart disease.







Categorical Feature Analysis

We analyzed categorical variables to see how different categories relate to heart disease. The categorical features analyzed include:

- Sex

- Chest pain type

- Exercise angina

- Slope of ST

<u>code:</u>

sns.countplot(data=df, x=feature, hue="Heart Disease")

plt.xticks(rotation=45) rotating the category labels on the x-axis improves readability.

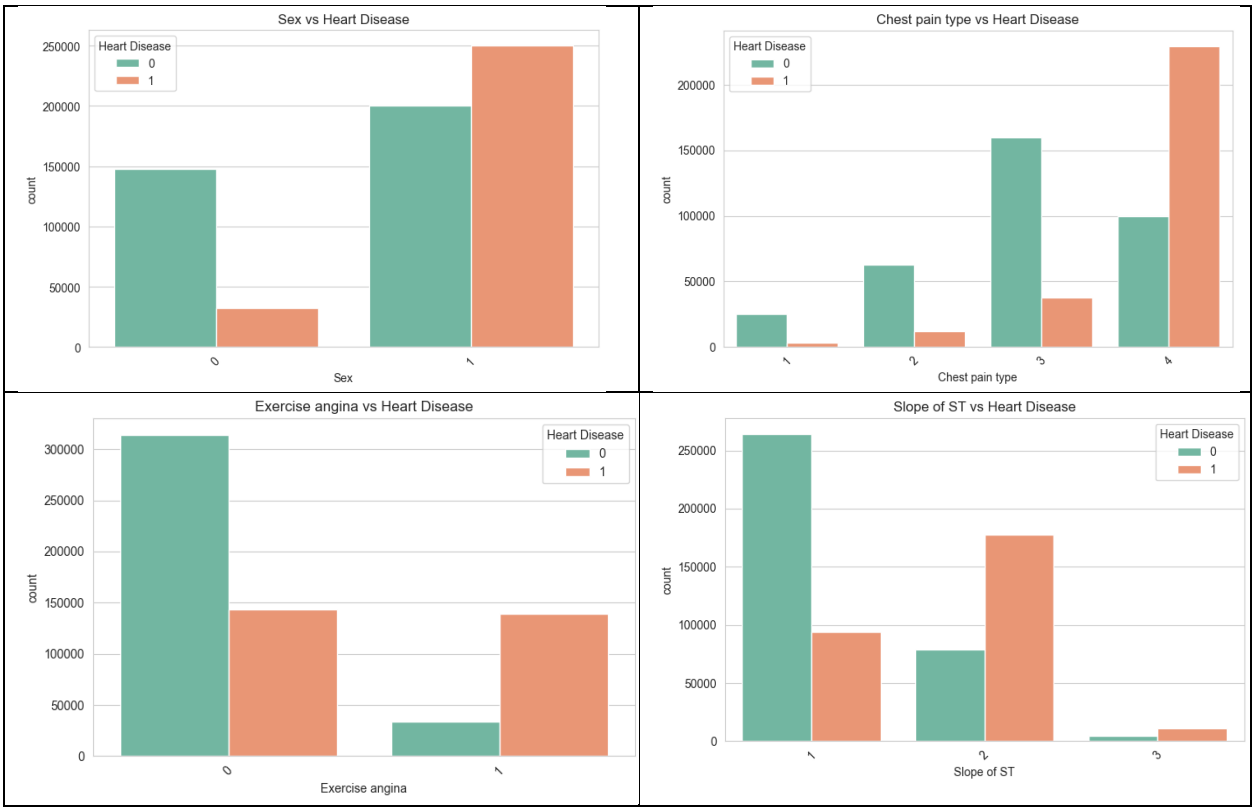
<u>Observation</u>

The analysis of categorical features indicates that some categories show stronger relationships with heart disease cases. For example:

- Male patients seem to have a higher frequency of heart disease.

- Certain chest pain types are more closely associated with heart disease.

- Patients with exercise angina have a higher risk of heart disease.



Correlation Analysis

To examine relationships between numerical health features and heart disease, we created a correlation matrix.

Before performing correlation analysis, we converted the target variable to numerical form:

df["Heart Disease"] = df["Heart Disease"].map({"Absence":0,"Presence":1})

This conversion is necessary because machine learning algorithms require numerical input. The numerical features used in the correlation analysis include:

- Age

- BP

- Cholesterol

- Max HR

- ST depression

- Number of vessels fluro

The correlation matrix was calculated using:

```
corr_matrix = df[numerical_features + ["Heart Disease"]].corr()
```

Correlation values range from -1 to +1:

Value	Meaning
+1	Strong positive relationship
0	No relationship
-1	Strong negative relationship

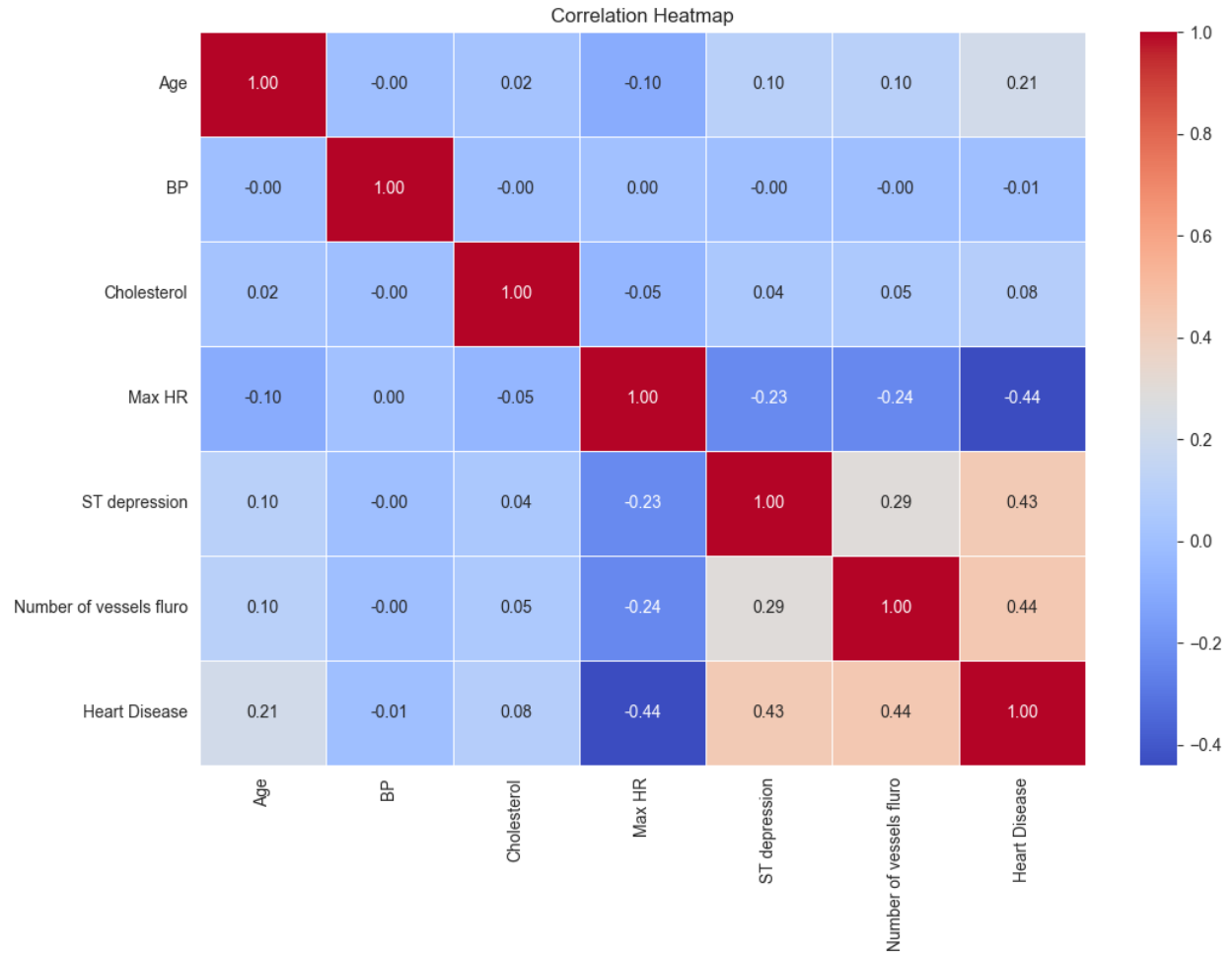
Heatmap Visualization

To visualize correlations, we created a heatmap using Seaborn.

```
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)
```

Explanation

annot=True displays correlation values inside each cell. cmap="coolwarm" sets the color scheme; red indicates positive correlation and blue indicates negative correlation. fmt=".2f" formats the values to two decimal places. This visualization helps identify which medical features are most strongly related to heart disease.




```
Heart Disease             1.000000
Number of vessels fluro   0.438604
ST depression             0.430641
Age                       0.212091
Cholesterol               0.082753
BP                       -0.005181
Max HR                   -0.440985
Name: Heart Disease, dtype: float64
```

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a technique used to reduce the complexity of datasets. PCA condenses many variables into a smaller number of components while maintaining the most important information.

Before applying PCA, we normalized the dataset using StandardScaler, which scales features to have similar ranges.

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

The dataset was divided into:

X → input features (health measurements)

y → target variable (Heart Disease)

We then applied PCA to reduce the dataset to two principal components for visualization.

pca = PCA(n_components=2)

This allowed us to visualize the dataset in two dimensions.

PCA Visualization

We created a scatter plot to visualize the PCA results. Each point represents a patient. Green points indicate patients without heart disease. Red points indicate patients with heart disease. The axes represent the two principal components:

- PC1

- PC2

These components capture the most important variations in the dataset. Sampling was applied to reduce the number of points plotted, making the visualization clearer for large datasets.

Conclusion

In this phase of the project, we conducted exploratory data analysis to understand the dataset and identify patterns related to heart disease.

Key observations include:

- The dataset contains no missing or duplicate values.

- The target variable shows slight class imbalance.

- Several numerical and categorical features show relationships with heart disease.

- Correlation analysis helped identify important medical attributes.

- PCA visualization demonstrated patterns within the dataset.

These insights will guide the development of machine learning models in the next phase of the project.